Informática

Tema 1: Lenguajes de programación



Docente 25-26: Daniel Castro Pereiro

Apuntes originales de David Mera Pérez y Efrén Arias Jordán

Área de Lenguajes y Sistemas
Dpto Electrónica e Computación

¿Qué es la Informática?

Información
+
auto**mática**
⇓
Informática

”Conjunto de conocimientos científicos y técnicas que hacen posible el tratamiento automático de la información por medio de ordenadores”

Ciencia que trata de la adquisición, almacenamiento, representación, tratamiento y transmisión de la **información**, utilizando máquinas denominadas ordenadores o computadoras.

Ordenador, Computadora, Computador:

Perspectiva física

- Máquina compuesta por elementos físicos (electrónicos, mecánicos, ...) capaz de realizar una serie de operaciones aritméticas, lógicas y de almacenar datos

Perspectiva lógica

- Máquina capaz de aceptar datos de entrada, procesarlos y proporcionar la información resultante a través de un medio de salida, sin interacción de un usuario, mediante un programa de instrucciones

Hardware

- También llamado soporte físico
- Conjunto de dispositivos físicos que componen el ordenador

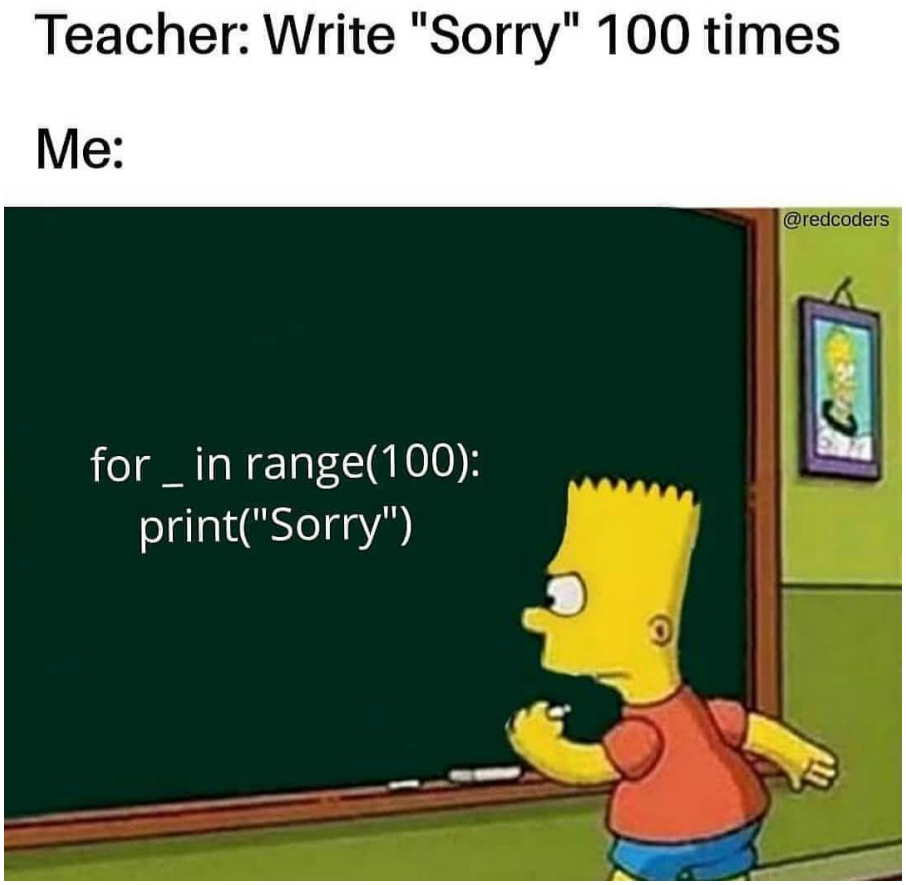
Software

- También llamado soporte lógico
- Conjunto de componentes intangibles de un ordenador
- Consiste en los procedimientos y datos para realizar distintas tareas

Representación de la información

¿Qué es un programa (informático)?

"Secuencia/conjunto de instrucciones que nos permiten realizar una tarea específica a través de un computador"



Representación de la información

Programación (informática)

- Proceso de crear conjuntos de instrucciones (programas) que permiten realizar algún tipo de tarea a través de un computador
- Parte de la Informática dedicada al estudio de las metodologías, algoritmos y lenguajes para desarrollar programas

Un algoritmo (informático) es ...

- Secuencia de instrucciones ordenadas, no ambiguas, cada una de las cuales requiere una cantidad finita de memoria y se realiza en un tiempo finito que permiten solucionar un problema.
- Un programa puede contener un conjunto de algoritmos

Software

- Conjunto de programas y otros datos relacionados

Información vs Datos

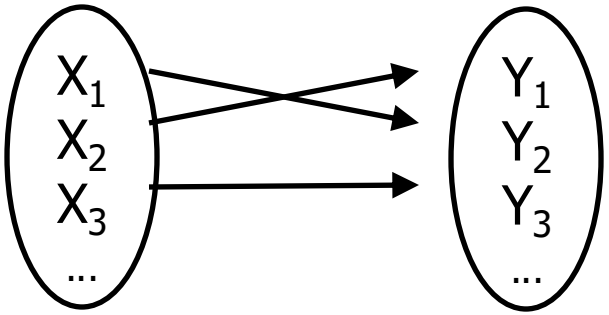
- Números, imágenes, símbolos, instrucciones, etc. sin un significado concreto y que no permiten tomar decisiones son datos.
- El procesado de datos nos permite dotarlos de significado y convertirlos en información

Representación interna de los datos

- Están expresados en un código que el ordenador es capaz de interpretar

Codificación:

- Transformación que representa los elementos de un conjunto mediante los elementos de otro



Representación de la información

Nº	=	Nº Romano
1	=	I
2	=	II
3	=	III
4	=	IV
5	=	V
6	=	VI
7	=	VII
8	=	VIII
9	=	IX
10	=	X
11	=	XI
12	=	XII
13	=	XIII
14	=	XIV
15	=	XV
16	=	XVI
17	=	XVII
18	=	XVIII
19	=	XIX
20	=	XX
21	=	XXI
22	=	XXII
23	=	XXIII
24	=	XXIV
25	=	XXV

Nº	=	Nº Romano
26	=	XXVI
27	=	XXVII
28	=	XXVIII
29	=	XXIX
30	=	XXX
31	=	XXXI
32	=	XXXII
33	=	XXXIII
34	=	XXXIV
35	=	XXXV
36	=	XXXVI
37	=	XXXVII
38	=	XXXVIII
39	=	XXXIX
40	=	XL
41	=	XLI
42	=	XLII
43	=	XLIII
44	=	XLIV
45	=	XLV
46	=	XLVI
47	=	XLVII
48	=	XLVIII
49	=	XLIX
50	=	L

Nº	=	Nº Romano
51	=	LI
52	=	LII
53	=	LIII
54	=	LIV
55	=	LV
56	=	LVI
57	=	LVII
58	=	LVIII
59	=	LIX
60	=	LX
61	=	LXI
62	=	LXII
63	=	LXIII
64	=	LXIV
65	=	LXV
66	=	LXVI
67	=	LXVII
68	=	LXVIII
69	=	LXIX
70	=	LXX
71	=	LXXI
72	=	LXXII
73	=	LXXIII
74	=	LXXIV
75	=	LXXV

Nº	=	Nº Romano
76	=	LXXVI
77	=	LXXVII
78	=	LXXVIII
79	=	LXXIX
80	=	LXXX
81	=	LXXXI
82	=	LXXXII
83	=	LXXXIII
84	=	LXXXIV
85	=	LXXXV
86	=	LXXXVI
87	=	LXXXVII
88	=	LXXXVIII
89	=	LXXXIX
90	=	XC
91	=	XCI
92	=	XCII
93	=	XCIII
94	=	XCIV
95	=	XCV
96	=	XCVI
97	=	XCVII
98	=	XCVIII
99	=	XCIX
100	=	C

Nº	=	Nº Romano
200	=	CC
300	=	CCC
400	=	CD
500	=	D
600	=	DC
700	=	DCC
800	=	DCCC
900	=	CM
1000	=	M
1100	=	MC
1200	=	MCC
1300	=	MCCC
1400	=	MCD
1500	=	MD
1600	=	MDC
1700	=	MDCC
1800	=	MDCCC
1900	=	MCM
2000	=	MM
2100	=	MMC
2200	=	MMCC
2300	=	MMCCC
2400	=	MMCD
2500	=	MMD
2600	=	MMDC

http://www.modelines.com

Para números igual o superior a 4000, se coloca una linea horizontal por encima del número, para indicar a la base la multiplicación por 1000

$\bar{V}$	5.000
$\bar{X}$	10.000
$\bar{L}$	50.000
$\bar{C}$	100.000
$\bar{D}$	500.000
$\bar{M}$	1000.000

Representación de la información

NÚMEROS EN JAPONÉS

○	0 zero	十一	11 juu ichi	百	100 hyaku
一	1 ichi	十二	12 juu ni	千	1.000 sen
二	2 ni	十三	13 juu san	万	10.000 man
三	3 san	十四	14 juu shi	百万	1.000.000 hyaku man
四	4 shi	二十	20 ni-juu	億	100.000.000 oku
五	5 go	二一	21 ni-juu ichi	兆	1.000.000.000.000 choo
六	6 rok	三十	30 san-juu		
七	7 nana	四十	40 shi-juu		
八	8 achi				
九	9 kyuu				
十	10 juu				

www.alfabetos.net

Representación de la información

Sociedad

- Empleamos un conjunto de símbolos (alfabeto) para crear palabras y frases asociadas a conceptos que nos permite expresarnos y transmitir información
- Para transmitir información numérica empleamos símbolos numéricos (normalmente en base 10)

Código Binario

- Sistema de numeración posicional en base 2 {0,1} empleado por los ordenadores para codificar los datos
- ¿Por qué binario? Cualquier ordenador está basado en dispositivos físicos (electrónicos), con los que es fácil determinar dos estados
  - encendido/apagado
  - voltaje positivo/negativo
  - magnetizado/desmagnetizado
  - ...

Decimal	Binario
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

Representación de la información

Representación de los datos en código binario

- El ordenador tendrá que transformar/codificar a código binario los datos para poder procesarlos y almacenarlos
- Denominamos **bit** a un dígito (0 o 1) del sistema de numeración binario y que representa la unidad mínima de información que podemos almacenar en un ordenador
- Con un bit solo podemos representar 2 valores por lo que es muy limitado
- Lo normal es que las memorias digitales tengan una capacidad de representación de 8, 16, 32, 64, …, bits.
- A través de secuencias de bits se pueden codificar números, palabras, imágenes, etc.

Agrupaciones de bits

- La memoria de un ordenador se mide en bits (se emplean múltiplos de bits)

1 byte	**8 bits**
1 Kilobyte (KB)	1024 bytes
1 Megabyte (MB)	1024 Kilobytes
1 Gigabyte (GB)	1024 Megabytes
1 Terabyte (TB)	1024 Gigabytes
1 Petabyte (PB)	1024 Terabytes
1 Exabyte (EB)	1024 Petabytes

1 bit	
1 Kilobit (Kb)	1024 bits
1 Megabit (Mb)	1024 kilobits
1 Gigabit (Gb)	1024 megabits
…	…

- Es habitual encontrar estos términos (dependiendo el contexto) como representaciones de múltiplos de 10

Representación de la información

Representación de los datos en código binario

- Los tipos de datos que se van a representar se resumen en los siguientes:

Números

- Números enteros, racionales, reales,. . .
- Según la representación (número bits) empleada
  - Rango de los números
  - Precisión

Caracteres

- Se utilizan códigos (numéricos) especiales (ASCII, UTF-8 …)
- Problemas con diferentes alfabetos

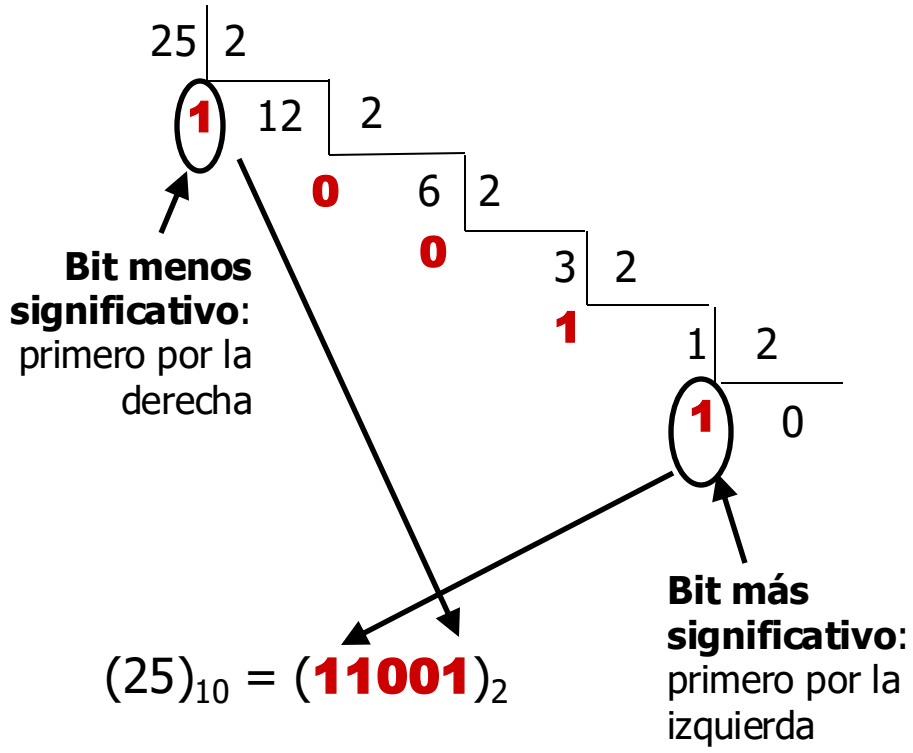
Instrucciones

- No son números: no tienen signo ni valor.
- Formadas por varios campos independientes: código de operación, registros de operandos, registro destino, . . .
- Cada campo se codifica de forma separada.
  - Si son números (p.e. un número de registro), se codifica como un entero.
  - Si es un código de operación, como una secuencia de bits con significado particular en este contexto.

Representación de la información

Conversión de decimal a binario

- Ejemplo: convertir 25 a binario
  - I. Se divide entre la base B hasta que el cociente sea 0
  - II. Se agrupan los restos de todas las divisiones.



Ejemplos

- Convertir a binario los siguientes números decimales:
  - 8
  - 15
  - 34

Conversión de binario a decimal

- Los números binarios están basados en potencias de 2 por lo que podemos convertirlos en base al contenido (0 o 1) y la posición de cada dígito dentro del número
- $11001 = 1 * 2^4 + 1 * 2^3 + 0 * 2^2 + 0 * 2^1 + 1 * 2^0 = 16+8+1=25$
- Ejemplos: Convertir a decimal los siguientes números
  - 111
  - 10110

Representación de la información

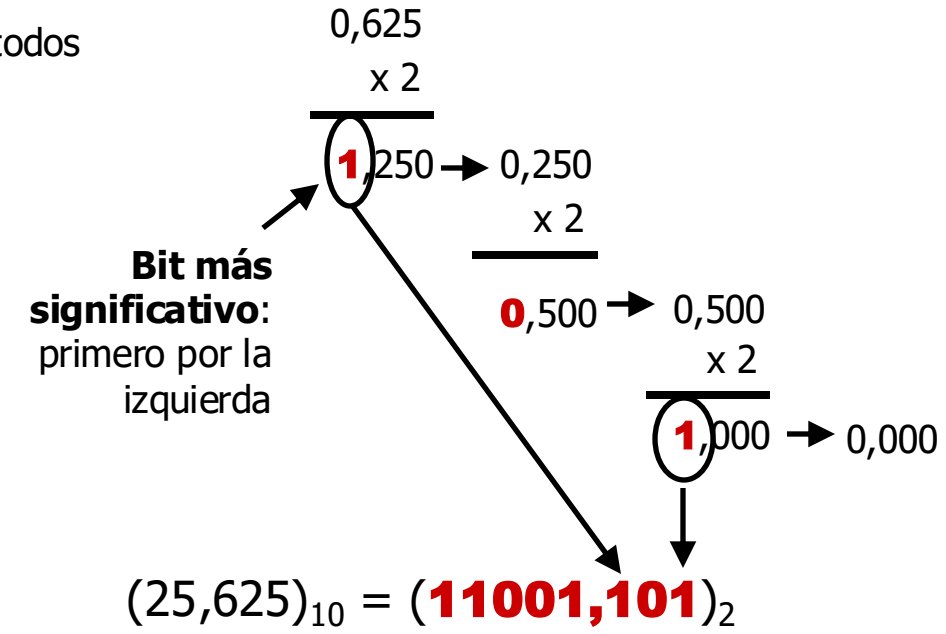
Conversión de decimales

- Ejemplo: convertir 25,625 a binario
- 25=11001 (calculado antes)
- La parte decimal se calcula aparte

I. Se multiplica la parte decimal por la base B y al resultado se le resta la parte entera

II. Se repite el paso I hasta que el resultado del producto sea 0

III. Se reúnen las partes enteras de todos los productos realizados



Representación de la información

Aritmética Binaria

- La codificación en sistema binario en los ordenadores facilita la realización de operaciones aritméticas y lógicas. Estas operaciones se hacen directamente sobre el dato binario. No es preciso realizar la transformación a decimal.

SUMA	**RESTA**
0 + 0 = 0 0 + 1 = 1 1 + 0 = 1 1 + 1 = 0 y llevo 1	0 – 0 = 0 0 – 1 = 1 y debo 1 1 – 0 = 1 1 – 1 = 0
PRODUCTO	**DIVISION**
0 · 0 = 0 0 · 1 = 0 1 · 0 = 0 1 · 1 = 1	0 : 0 = NaN 0 : 1 = 0 1 : 0 = ∞ 1 : 1 = 1

- El código binario permite definir las mismas operaciones aritméticas y lógicas que el decimal. Las anteriores operaciones se pueden generalizar fácilmente números binarios de cualquier longitud

Operaciones lógicas

- Los operadores lógicos se usan **en lógica proposicional** para admitir o rechazar proposiciones
- Los operandos son valores lógicos: VERDADERO / FALSO
- En informática se representan de la siguiente forma
  - VERDADERO ⇒ 1
  - FALSO ⇒ 0
- Operadores lógicos

O/OR	**Y/AND**	**NO/NOT**
0 OR 0 = 0 0 OR 1 = 1 1 OR 0 = 1 1 OR 1 = 1	0 AND 0 = 0 0 AND 1 = 0 1 AND 0 = 0 1 AND 1 = 1	NOT 0 = 1 NOT 1 = 0

Representación de la información

Codificación Hexadecimal

- Por comodidad, se usa representación hexadecimal
  - Base 16, Dígitos **0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F**
  - A=10, B=11, C=12, D=13, E=14, F=15
- La conversión es sencilla:

4 bits => 1 dígito hexadecimal

Hexadecimal	Binario
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111

Hexadecimal	Binario
8	1000
9	1001
A	1010
B	1011
C	1100
D	1101
E	1110
F	1111

Ejemplo:

00111010101100001111110100001000 =

0011 1010 1011 0000 1111 1101 0000 1000 =

0x3AB0FD08

Decimal	Hexadecimal	Binario
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
10	A	1010
11	B	1011
12	C	1100
13	D	1101
14	E	1110
15	F	1111

Representación de la información

De decimal a hexadecimal

- Igual que binario pero dividiendo entre 16
  - Vemos el paso del número 110 en decimal a hexadecimal

110	16
14	6

Luego, el número 110 en decimal, sería **6E** en hexadecimal (E=14)

Ejercicio: Convertir de decimal a hexadecimal

- 847
- 2570

De hexadecimal a decimal

- Igual que binario pero utilizando 16 como base
  - Vemos el paso del número 6E en hexadecimal a decimal

$6*16^1+E*16^0= 6*16^1+14*16^0=110$

Ejercicio: Convertir de hexadecimal a decimal

- AA
- 34D

De binario a hexadecimal

- Agrupar de 4 en 4 comenzando por la derecha. Convertir cada grupo (número binario) a un dígito hexadecimal
  - Vemos el número 101101
    - Grupo 1101 se convierte en D (D=13)
    - Grupo 10 se convierte en 2

Luego, el número 101101 en binario sería **2D** en hexadecimal

Ejercicio: Convertir de binario a hexadecimal

- 1001111 101100

De hexadecimal a binario

- Representar cada dígito hexadecimal en binario
  - Vemos el número 2D
    - El 2 en binario sería 10
    - El D es el 13 en decimal y esto es el 1101 en binario

Luego, el número 2D en hexadecimal sería 101101 en binario

Ejercicio: Convertir de hexadecimal a binario

- F8 4E9

Representación de la información

Representación de caracteres

- Código ASCII: 7 bits ⇒ hasta 128 caracteres
  - Creado en los años 60, incluye sólo el alfabeto inglés y caracteres de control
  - Cada carácter se representa mediante un entero de 7 bits

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[ENG OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	|
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

Representación de la información

Extended ASCII (8 bits)

- Extienden ASCII con otros 128 caracteres
  - 1 byte por carácter (256 caracteres en total)
  - Los 128 primeros (0-127) iguales que ASCII
- ISO/IEC 8859
  - **15 estándares** para diferentes idiomas
  - **8859-1 (latin-1)** Europa occidental, 8859-4 (latin-4) Europa norte,
  - 8859-7 Griego,. . .
- Windows-1252 (o CP-1252)
  - Cambia algunos caracteres de control del 8859-1 por caracteres imprimibles
- Mac OS Roman
  - Usada en Mac OS

iso-8859-1

+	0	1	2	3	4	5	6	7	8	9
160		¡	¢	£	¤	¥	¦	§	¨	©
170	ª	«	¬	-	®	¯	°	±	²	³
180	´	µ	¶	·	¸	¹	º	»	¼	½
190	¾	¿	À	Á	Â	Ã	Ä	Å	Æ	Ç
200	È	É	Ê	Ë	Ì	Í	Î	Ï	Ð	Ñ
210	Ò	Ó	Ô	Õ	Ö	×	Ø	Ù	Ú	Û
220	Ü	Ý	Þ	ß	à	á	â	ã	ä	å
230	æ	ç	è	é	ê	ë	ì	í	î	ï
240	ð	ñ	ò	ó	ô	õ	ö	÷	ø	ù
250	ú	û	ü	ý	þ	ÿ				

iso-8859-4

+	0	1	2	3	4	5	6	7	8	9
160		Ą	ĸ	Ŗ	¤	Ĩ	Ļ	§	¨	Š
170	Ē	Ģ	Ŧ	-	Ž	¯	°	ą	˛	ŀ
180	´	ĩ	ļ	ˇ	¸	š	ē	ģ	ŧ	ŋ
190	ž	ŋ	Ā	Á	Â	Ã	Ä	Å	Æ	Į
200	Č	É	Ę	Ë	Ė	Í	Î	Ī	Đ	Ņ
210	Ō	Ķ	Ô	Õ	Ö	×	Ø	Ų	Ú	Û
220	Ü	Ũ	Ū	ß	ā	á	â	ã	ä	å
230	æ	ļ	č	é	ę	ë	ė	í	î	ī
240	đ	ņ	ō	ķ	ô	õ	ö	÷	ø	ų
250	ú	û	ü	ũ	ū	˙				

iso-8859-7

+	0	1	2	3	4	5	6	7	8	9
160		‘	’	£			¦	§	¨	©
170		«	¬	-		-	°	±	²	³
180	΄	΅	Ά	·	Έ	Ή	Ί	»	Ό	½
190	Ύ	Ώ	ΐ	Α	Β	Γ	Δ	Ε	Ζ	Η
200	Θ	Ι	Κ	Λ	Μ	Ν	Ξ	Ο	Π	Ρ
210		Σ	Τ	Υ	Φ	Χ	Ψ	Ω	Ϊ	Ϋ
220	ά	έ	ή	ί	ΰ	α	β	γ	δ	ε
230	ζ	η	θ	ι	κ	λ	μ	ν	ξ	ο
240	π	ρ	ς	σ	τ	υ	φ	χ	ψ	ω
250	ϊ	ϋ	ό	ύ	ώ					

Representación de la información

Unicode

- Estándar para la representación de texto en la mayoría de los idiomas mundiales
- Incluye caracteres gráficos (letras, signos diacríticos, cifras,...) en múltiples idiomas, símbolos de puntuación, códigos de control, etc.
- Se codifica en octetos (bytes)
- Los caracteres de ISO-8859-1 son los primeros 256 caracteres del standard

Unicode: Formas de codificación

UTF-8

- Es capaz de representar cualquier carácter Unicode
- Usa símbolos de longitud variable (de 1 a 4 bytes por carácter Unicode).
- Incluye la especificación ASCII, por lo que cualquier mensaje ASCII se representa sin cambios

UTF-16

- Utiliza símbolos de longitud variable, 1 o 2 palabras de 16 bits por carácter Unicode (2 o 4 bytes).
- La unidad de información es la palabra de 16 bits.
- La mayoría de los caracteres se codifican con 16 bits
- Transformación adicional de la codificación Unicode

UTF-32

- Utiliza exactamente 32 bits (4 bytes) por carácter Unicode
- Su principal ventaja es la sencillez
- Su desventaja es la ineficiencia en el uso de espacio de almacenamiento

Programas y Lenguajes de Programación

Objetivos

- **¿Qué vamos a aprender en esta materia?**
  - Partiendo de un problema sencillo, ser capaz de encontrar un algoritmo que lo resuelva
  - Desarrollar programas sencillos
  - Fundamentos de programación que hay detrás
  - Conceptos básicos, comunes a todos los lenguajes
  - Metodología para desarrollar programas de forma sistemática y eficiente

- Un lenguaje algorítmico sencillo ejemplificándolo en un lenguaje de programación muy potente: **Python**

```
#Programa para calcular la suma de dos numeros float
#Pedimos los datos por teclado

a=float(input("Dame a --> "))
b=float(input("Dame b --> "))

#Calculamos la suma
res = a + b

#Mostramos el resultado
print("La suma es --> ", res)
#Fin
```

Definiciones previas:

- **Archivo** o **Fichero:** conjunto de datos almacenados en disco como una entidad individual e identificados por un nombre.
- **Programa:** Un fichero (o zona de memoria) que contiene una secuencia ordenada de instrucciones que le indican a un ordenador las operaciones o tareas a realizar.
- **Rutina**, **procedimiento o función:** conjunto de instrucciones o subprograma que utiliza el programa principal sólo cuando se considera necesario para realizar una tarea específica.

Programas y Lenguajes de Programación

- Un **algoritmo** es una secuencia ordenada de operaciones tal que su ejecución resuelve un determinado problema. Un algoritmo debe ser:
  - **Preciso:** indicar detalladamente cada paso y el orden de realización de cada uno.
  - **Definido:** bajo las mismas condiciones de partida, el algoritmo siempre producirá el mismo resultado
  - **Finito:** debe tener un número finito de pasos
  - **Independiente del lenguaje de programación** que se utilice para la codificación

- **Algoritmo: Estados + Acciones.**

Secuencia de instrucciones tal que, seguidas paso a paso, proporciona una secuencia ordenada de estados finita en la que el último estado (o estado final) contiene la solución del problema a resolver para cada entrada válida.

- **Ejemplo**: Algoritmo de Euclides para el cálculo del máximo común divisor de dos números
  - **Paso 1**. Tomar el número mayor como dividendo y el menor como divisor.
  - **Paso 2.** Calcular el resto de la división entera.
  - **Paso 3**. Si el resto es igual a 0 entonces ir al Paso 4. En caso contrario, tomar el divisor y el resto y volver al Paso 1.
  - **Paso 4**. El m.c.d. es el divisor de la última división.

Programas y Lenguajes de Programación

- La **Programación** consiste desarrollar el conjunto de instrucciones que ha de ejecutar el procesador para realizar una determinada tarea

- Los procesos necesarios para solucionar un problema (realizar una tarea) mediante un ordenador:
  - **Especificación** y análisis del problema que queremos solucionar
  - **Diseño** del algoritmo
  - **Codificación** (en un lenguaje de programación)
  - **Prueba** del código
  - **Mantenimiento** (resolución de problemas, actualización, ...)

- Fases de la programación (desde el punto de vista de escritura código)
  - **Problema** (lenguaje natural)
  - ⇩
  - **Algoritmo** (Pseudocódigo)
  - ⇩
  - **Codificación** (lenguaje de programación concreto)
  - ⇩
  - **Compilación** (lenguaje máquina)/ Interpretación
  - ⇩
  - **Ejecución**

Programas y Lenguajes de Programación

- La codificación de programas se realiza mediante **lenguajes de programación**. Es el código en el que se expresan las instrucciones de los programas

Ejemplo: programa en C++ que lea dos números reales, calcule su suma y la muestre en pantalla.

```cpp
#include <iostream>

using namespace std;

/*
Programa para calcular la suma de dos números flotantes
*/

int main (){
    float a, b, res=0;

    // pedir los datos por teclado
    cout << "Dame a--> " ;
    cin >> a;
    cout << "Dame b--> ";
    cin >> b;

    // calcular la suma
    res = a + b;

    // mostrar el resultado
    cout << "La suma es-->" << res << endl;

    // finalizar
    return 0;
}
```

Como se hace?

- Secuencia de pasos:
  - pedir los datos por teclado
  - calcular la suma
  - mostrar el resultado

Ejemplo: el mismo programa en Python

```python
#Programa para calcular la suma de dos numeros float
#Pedimos los datos por teclado

a=float(input("Dame a --> "))
b=float(input("Dame b --> "))

#Calculamos la suma
res = a + b

#Mostramos el resultado
print("La suma es --> ", res)
#Fin
```

Programas y Lenguajes de Programación

- Diferentes niveles de abstracción:
  - **Lenguaje máquina**: Utiliza un **código binario** para representar las instrucciones elementales, directamente interpretables por el procesador.
  - **Lenguaje de Programación de Bajo Nivel:** La codificación de las instrucciones es compleja y dificultosa para un humano. **Lenguaje ensamblador:** versión "legible" del lenguaje máquina
  - **Lenguajes de programación de alto nivel:** la codificación de las instrucciones es más sencilla y comprensible para un humano, se emplea una sintaxis similar al lenguaje natural (simplificado y típicamente términos en inglés).

Lenguaje Máquina

- Utiliza un **código binario** para representar las instrucciones elementales, directamente interpretables por el procesador. Cada procesador tiene su propio **repertorio de instrucciones.** Cada instrucción se representa por un valor del código **binario**
- Esta codificación de las instrucciones constituye el **lenguaje máquina** de un procesador
- Un fichero de código en lenguaje máquina es directamente **ejecutable** por el correspondiente procesador. Distintos tipos de procesadores utilizan distintos lenguajes máquina
- Problemas:
  - Dificultad y lentitud en la codificación y verificación
  - **No es portable**: el código máquina de un procesador no se puede ejecutar en otro tipo de procesador

Programas y Lenguajes de Programación

Tipos de instrucciones elementales que puede ejecutar un procesador

- **instrucciones de transferencia de datos**.
  - El objetivo es transferir datos entre la memoria, la ALU y los dispositivos de entrada y salida.
- **instrucciones aritmético-lógicas**
- **instrucciones de control de flujo (saltos)**
  - Este tipo de instrucciones permiten alterar el orden secuencial de ejecución de una secuencia de instrucciones.
    - **Condicionales o bifurcaciones**: permiten realizar acciones alternativas en función de una condición
    - **Bucles o repeticiones**: permiten repetir una operación un número determinado de veces sobre distintos datos.

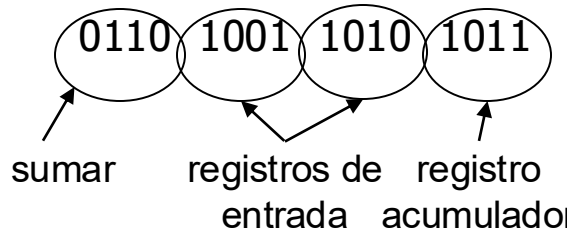
Lenguaje de Programación de Bajo Nivel

- Fueron creados para evitar usar el código binario para programar
- Se acercan más a los lenguajes humanos (inglés)
- También son **dependientes del procesador**
- **El lenguaje Ensamblador:**
  - Usa nombres **mnemotécnicos** para las instrucciones del procesador
  - **Requiere un proceso de "traducción"**: Para poder ejecutarlos hay que convertir el código ensamblador a código máquina

Realizar la operación z = x + y

código ensamblador: **ADD x, y, z**

código máquina:

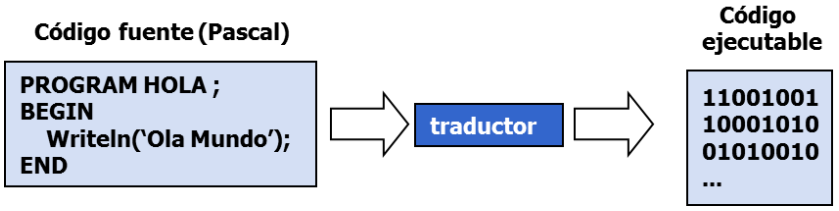


Programas y Lenguajes de Programación

Lenguajes de Programación de Alto Nivel

- Incorporan **instrucciones de alto nivel**:
  - instrucciones más complejas, que son composición de las instrucciones elementales del procesador
  - Ej. x = coseno(y); En realidad son subprogramas o rutinas que se pueden ejecutar desde otros programas.
- El **proceso de "traducción"** se encarga de convertir el código de alto nivel en código interpretable por el procesador correspondiente (lenguaje máquina)
- **Portabilidad:** Los lenguajes de alto nivel no dependen del hardware, se pueden traducir y ejecutar en equipos con diversos procesadores. Intentan parecerse al lenguaje humano/matemático para simplificar la codificación
  - **Ejemplos**: C, C++, Fortran, Pascal, Java, Visual Basic, ...

- Los lenguajes de alto nivel se pueden clasificar en **compilados** o **interpretados** según el tipo de traducción que utilizan
  - **Código fuente**: programa expresado en un lenguaje de programación distinto del código máquina
  - **Código ejecutable:** programa expresado en código máquina
- El siguiente ejemplo presenta el código fuente de un programa en lenguaje Pascal



Programas y Lenguajes de Programación

Proceso de "traducción" en lenguajes interpretados

- Es un traductor-ejecutor: a medida que traduce cada instrucción la va ejecutando.
- El código máquina resultante no se almacena en un fichero
- Para ejecutar el programa siempre necesitamos el intérprete
- El fichero fuente se llama **script**. Ejemplos de lenguajes interpretados (o de script)

  Matlab, Lisp, Perl, Python, JavaScript, VBScript, PHP, ASP, …
- Un programa puede estar compuesto de uno o varios scripts

- Para cada programa se realizan los siguientes pasos
  1. Se carga la primera instrucción
  2. Se traduce
  3. Se ejecuta
  4. Se determina la siguiente instrucción a ejecutar (puede estar en el mismo script o en otro)
  5. Se vuelve al paso nº 2

Programas y Lenguajes de Programación

Proceso de "traducción" en lenguajes compilados

- El código fuente se analiza en su totalidad por un compilador que traduce el código fuente a código máquina (binario)
- El código máquina resultante se almacena en un fichero ejecutable.
- No se necesita compilador para la ejecución del programa
- Típicamente son programas más eficientes que los interpretados
- El ejecutable es dependiente de la arquitectura del ordenador
- Ejemplos de **lenguajes compilados**:

  Pascal, C, C++, Fortran, Basic, Visual Basic, Java, Ada, Cobol, ...

Entorno de desarrollo

- **Software de programación:** software para escribir programas utilizando lenguajes de programación y para generar código ejecutable a partir de ellos**.** Un **entorno integrado de desarrollo (IDE)** incorpora las herramientas necesarias para todas las etapas de desarrollo.
  - **Codificación**: editor de código, librerías estándar, documentación del lenguaje
  - **Compilación/interpretación**:
  - **Ejecución**
  - **Depuración de código**
    - Cuando se producen errores de compilación se inicia un proceso de depuración del código
    - **Debugger**: utilidad que permite ejecutar el código instrucción a instrucción e inspeccionar el estado de las variables en cada momento